BookScope

Explorer le catalogue Goodbooks-10k par les liens entre lecteurs et livres

Amine Kerfai et Abdelhamid Ouazzou

M2 SLCD · Big Data and NoSQL · Projet 2 · novembre 2026

1. Présentation

BookScope est une application Java qui explore le catalogue Goodbooks-10k à partir des notes des lecteurs. Elle propose les six services du sujet, F1 à F6, en ligne de commande et dans une petite interface web.

Le même code tourne sur deux déploiements : un serveur MongoDB seul (partie I), puis un cluster de deux shards sur la même machine (partie II). Seule l'adresse de connexion change. Une troisième version du cluster, dans des conteneurs Docker, rapproche un peu plus la mesure d'un vrai réseau.

Toutes les mesures viennent d'un Windows 11 à 8 cœurs et 16 Go de mémoire, avec MongoDB 9.0.2 (cache de 0,5 Go par instance), Java 17 et le pilote mongodb-driver-sync 5.1.4. Le README décrit les commandes pour tout reproduire.

2. Données

2.1 Sélection

Nous gardons les lecteurs d'identifiant 30000 à 39999, soit 10 000 lecteurs sur 53 424, avec tout leur historique.

Ce choix agit sur les liens que l'on voit. Deux livres reliés surtout par des lecteurs hors de la tranche paraissent moins liés qu'ils ne le sont, et le livre 7803 n'a plus aucune note. Les comptes de tags de book_tags.csv, eux, portent sur tout Goodreads et non sur nos lecteurs, ce qui compte pour E3.

2.2 Jeux mis de côté

Chaque lecteur est rangé dans un groupe, le SHA-1 de « bookscope :identifiant » modulo 100. Les groupes 0 à 19 forment le jeu d'évaluation de E3 : les 3 dernières notes de ces lecteurs, soit 6 012 notes. Les groupes 20 à 24 forment le lot de F6 : leurs 2 dernières notes, soit 1 058. Les lecteurs diffèrent d'un jeu à l'autre et ces notes ne sont chargées dans aucune base. Comme ratings.csv n'a pas de dates, l'ordre des lignes sert d'ordre dans le temps.

2.3 Chiffres

Élément	Valeur
Lecteurs	10 000
Notes stockées	1 084 334
Livres notés / catalogue	9 999 / 10 000
Entrées to_read	184 504
Tags / liens livre-tag	34 250 / 999 898
Notes par lecteur	min 19, médiane 108, p95 = 153, max 200
Notes par livre	min 1, médiane 46, p95 = 378, max 4 273

Table 1. Contenu de la sélection.

Les livres sont très inégaux : 46 notes pour le livre médian, 4 273 pour le plus noté. Les pondérations de la section 3 en tiennent compte. Les anomalies des fichiers sont en annexe.

3. Modèle de données

MongoDB convient bien ici : les données sont des documents simples (un livre avec ses tags, une note) et le même code tourne sur un serveur ou sur un cluster. Le serveur est un processus à part, interrogé par le réseau.

Collection	Contenu	Index et répartition
books	métadonnées et tags triés par compte	book_id ; non répartie, sur le shard primaire
tags	nom, nombre de livres, tag de statut ou non	tag_id, nom ; shard primaire
ratings	user_id, book_id, note, ordre	_id « lecteur :livre » ; (book, note, user) ; (user, note, book) ; (user, ordre) ; répartie par user_id
to_read	user_id, book_id, ordre	répartie par user_id
user_stats, book_stats	nombre de notes et somme	tenues à jour par F6 ; user_stats répartie par lecteur, book_stats sur le shard primaire
rating_changes	ancienne note remplacée	journal de F6

Table 2. Collections.

Une note est identifiée par le couple (lecteur, livre). Les deux index composés répondent aux deux questions les plus fréquentes : qui a aimé ce livre, et qu'a aimé ce lecteur. Les compteurs sont calculés au chargement puis tenus à jour par F6.

Trois règles servent partout. Une note est positive à partir de 4. Dans F2, un livre pèse le logarithme du rapport entre le nombre de lecteurs et son nombre de notes : s'accorder sur un livre très lu compte moins. Un tag est générique quand il décrit un statut (to-read, owned, favorites, read-in-2016) ou quand au moins la moitié des livres le portent ; il ne prouve jamais un lien.

4. Services

Chaque service reçoit ses entrées à l'exécution et calcule sa réponse sur les données stockées. Tout résultat porte les identifiants qui le justifient, et la commande records affiche les enregistrements correspondants. Pour le lecteur 32055 et le livre 7, elle donne la note 3 et les tags de The Hobbit avec leurs comptes.

	Entrée et sortie	Principe et garde-fous	Exemple réel
F1	Un titre ou un identifiant. Fiche du livre et livres liés.	Lecteurs communs (cosinus, lift) et tags spécifiques. Chaque lien dit sur quoi il repose : lectorat commun, tags spécifiques ou tags génériques seulement. Titre ambigu : liste, jamais de choix silencieux.	« the hobbit » donne 4 livres (7, 466, 964, 1129). Le livre 349 (‘Salem’s Lot) : 629 notes, 427 lecteurs l’ont aimé, It en tête des livres liés.
F2	Deux lecteurs. Livres communs, accords, désaccords, force du lien.	Corrélation de notes centrées par lecteur, pondérée contre la popularité, réduite si peu de livres communs. Sous 5 livres communs : insufficient_overlap.	32055 et 36002 : 84 livres communs, Jaccard 0,31, corrélation -0,04 (lien faible). Désaccord sur Bleak House (1 contre 5). 34590 et 39865 : 3 livres communs, pas de conclusion.
F3	Un lecteur, troncature k facultative. Liste classée avec explications.	50 lecteurs voisins, pénalité sur les livres populaires. Moins de 10 notes : 50 % voisins, 50 % profil de tags, avec mention de la troncature. Part du top 100 rapportée.	32055 tronqué à 5 notes : Pride and Prejudice en tête, aimé par 36 des 50 voisins.
F4	Deux livres. Verdict et chemins.	Lift des lecteurs communs contre l’indépendance, lecteurs actifs pondérés à la baisse, chemins à deux sauts en pénalisant le 1 % de livres les plus notés. Sinon : pas de connexion convaincante, avec raisons.	349 vers 7 : 137 lecteurs communs pour 103 attendus, lift 1,33, connexion directe.
F5	Deux tags. Livres communs et lecteurs attachés.	Un livre appartient à un intérêt si le tag figure dans ses 10 premiers, au moins 20 fois. Classement par lift sur les lecteurs, refait sans le 1 % de livres les plus notés et les lecteurs au-dessus du p95.	vampires et romance : New Moon (49), Eclipse (52), Dark Lover (433).
F6	Un fichier jsonl ou csv. Compteurs d’import.	Identité (lecteur, livre). Si la note change, la dernière importée gagne et l’ancienne va dans rating_changes. Lignes invalides rejetées avec motif.	Lot de 1 058 notes : 1 058 insertions, puis 0 insertion et 1 058 lignes inchangées au deuxième import.

Table 3. Les six services.

5. Partie I : représentation alternative

Le modèle de départ range une note par document. Nous l’avons confronté à un document par lecteur, qui contient toutes ses notes (10 000 documents au lieu de 1 085 392, soit les 1 084 334 notes chargées plus les 1 058 du lot de F6 importé avant la mesure). Les réponses sont identiques.

Médiane de 5 répétitions	Une note par document	Un document par lecteur	Par lecteur plus rapide pour
Historique d’un lecteur	1,42 ms	0,47 ms	95 lecteurs sur 100
Qui a aimé ce livre	0,70 ms	1,18 ms	12 livres sur 100
Taille (données, index)	73 Mo, 79 Mo	31 Mo, 7 Mo	-

Table 4. Comparaison des deux représentations.

La version par lecteur lit un historique trois fois plus vite et occupe bien moins de place, mais elle répond moins vite à « qui a aimé ce livre ». Nous avons gardé une note par document, pour trois raisons : F1, F4 et F5, les services les plus coûteux, partent des livres ; la lecture d’un historique ne pèse qu’environ 1 ms dans un F2 de 4 ms ; et F6 ajoute une note sans réécrire un gros document.

6. Partie II : déploiement distribué

6.1 Placement

Le cluster comprend un serveur de configuration, deux shards (ports 27211 et 27212, chacun avec son dossier de données) et un routeur mongos (port 27200) qui sert d'interface unique. Les collections ratings et to_read sont réparties par lecteur ; books, tags et book_stats ne sont pas répartis : ils restent sur le shard primaire (shard0), et le programme les lit une fois en mémoire. Le placement principal coupe user_id en deux plages qui contiennent autant de notes (coupure à 35004). L'historique d'un lecteur tient donc sur un seul shard, alors qu'une question sur un livre doit interroger les deux.

Requête	**Instances contactées**	**Travail local**	**Échanges**
Un lecteur (F2, historique de F3)	1, celle du lecteur	tout	aucun
Un livre (qui a aimé X)	les 2	filtrage par index	listes de lecteurs vers mongos
Parcours lecteurs puis livres (F4, F1)	les 2, deux fois	agrégation partielle	sous-totaux fusionnés par mongos

Table 5. Instances contactées (placement par plages).

6.2 Résultats identiques

compare-deployments envoie 14 requêtes aux deux déploiements, trie les résultats dans un ordre fixe avec un départage déterministe, puis compare. Les requêtes couvrent des titres ambigus, des lecteurs du même shard et de shards différents, un cas à peu de livres communs, des historiques tronqués, F3, F4 et F5. Les 14 résultats sont identiques entre le serveur seul et le cluster, et entre le cluster et Docker. La comparaison suppose les mêmes notes partout : après l'import du lot F6 sur un seul déploiement, presque toutes les requêtes différaient ; elles sont redevenues identiques une fois le même lot importé sur l'autre.

6.3 Seconde répartition et Docker

Nous avons essayé une seconde répartition : les lecteurs sont placés par hachage de user_id au lieu de plages, et un lecteur reste entier sur un shard. Ses mesures sont en 7.2. La version Docker place chaque instance dans un conteneur limité à 1 processeur et 1 Go, avec 2 ms de délai réseau sur chaque paquet sortant.

Médiane de 5 (ms)	**1 serveur**	**2 shards, même machine**	**2 shards, Docker**
F2, même shard	14	11	46
F2, deux shards	10	12	44
F3, lecteur 32055	283	234	432
F4, 349 vers 7	428	474	600
F5, vampires et romance	1 216	1 128	962

Table 6. Temps sur la sélection de 10 000 lecteurs.

Les requêtes courtes sont environ quatre fois plus lentes dans Docker : elles se résument à des allers-retours de 4 ms. Les requêtes longues, comme F5, ne ralentissent pas.

6.4 Perte d'une instance

Le shard 1 arrêté, F2 sur 32055 et 36002 puis F3 sur 32055 répondent status : unavailable, sans aucun résultat partiel : mongos refuse la requête (erreur 133). La réponse arrive après environ 16 s, sur le cluster local comme dans Docker ; dans Docker, une fois le conteneur redémarré, la même requête prend 1,1 s.

Nous ne récupérons rien tant que le shard est arrêté, car chaque shard n'a qu'un membre. En production, chaque shard serait répliqué sur plusieurs machines et un secondaire prendrait le relais. Notre réponse est plus sévère que nécessaire : F2 échoue même quand les deux lecteurs sont sur le shard en vie (32305 et 33065). La cause est le chargement du catalogue, qui compte tous les lecteurs dans user_stats, collection répartie : ce comptage a besoin des deux shards. Enregistrer ce nombre au chargement, puis le tenir à jour dans F6, permettrait à F2 de répondre pour deux lecteurs du shard en vie ; F3 échouerait toujours, car la recherche de voisins interroge tous les shards. Avec les plages, l'historique d'un lecteur est entier ou absent, jamais à moitié perdu. Enfin, ce n'est pas un cluster de machines indépendantes : processeur, mémoire, disque et alimentation sont partagés.

7. Expériences

7.1 E1. Taille et performances

Trois sous-ensembles de données réelles sont chargés sur le serveur seul : 10 000 lecteurs, 30 000 (identifiants 20000 à 49999) et tous. Chaque mesure rejoue 30 requêtes tirées au hasard (graine 42), cinq fois, après 10 requêtes d'échauffement. Le catalogue est déjà en mémoire et les serveurs ne sont pas redémarrés entre deux essais.

	Petit	**Moyen**	**Grand (tout)**
Lecteurs	10 000	30 000	53 424
Notes	1 084 334	3 273 049	5 939 285
Entrées to_read	184 504	541 282	912 705
Chargement	21 s	77 s	141 s
Taille (données et index)	133 Mo	281 Mo	494 Mo
F1, médiane	23 ms	29 ms	33 ms
F2, médiane	4,2 ms	4,2 ms	3,2 ms
F3, médiane	122 ms	264 ms	322 ms
F4, médiane	34 ms	52 ms	60 ms
F5, médiane	901 ms	2 989 ms	5 910 ms

Table 7. E1.

Le chargement et la taille suivent à peu près le nombre de notes. F2 reste stable, puisqu'elle ne lit que deux historiques. F3 est multipliée par 2,6 pour 5,5 fois plus de notes, et F5 par 6,6 : elle parcourt les lecteurs de chaque intérêt, puis recommence sans les cas populaires.

7.2 E2. Local contre distribué

Nous comparons, sur le jeu complet, trois configurations : un serveur seul (A), deux shards par plages (B) et deux shards par hachage (C). Avant les mesures, nous avions formulé la prédiction suivante.

Question	**Attendu**	**Raison**
Charge personnalisée (F2, F3)	A le plus rapide	F2 lit deux historiques, ciblés sur un seul shard : le partage ajoute un saut par mongos sans économiser de travail. F3 cherche des voisins par livre, ce qui devient une interrogation des deux shards suivie d'une fusion, sur des processus qui se partagent processeur et disque.
Charge d'exploration (F1, F4, F5)	B et C proches de A, peut-être un peu plus rapides	Ces requêtes agrègent les notes de nombreux lecteurs : chaque shard traite la moitié des données en parallèle, avec au total deux fois le cache. La fusion par mongos et le disque partagé limitent le gain.
Mélange des deux	L'avance de A diminue quand l'exploration progresse, et peut s'inverser	Même raison que pour l'exploration.
B contre C	Pas de différence notable	Aucune requête n'utilise des plages de user_id ; C répartit peut-être mieux le travail entre les shards.

Table 8. Prédiction.

La charge personnalisée utilise F2 et F3, la charge d'exploration F1, F4 et F5. Un lot compte 30 requêtes identiques pour les trois configurations (graine 42), rejouées cinq fois après échauffement, avec une part de requêtes personnalisées allant de 100 % à 0 %.

Part personnalisée	**1 serveur**	**Plages**	**Hachage**	**Écart-type**
100 %	7,3 s	8,1 s	8,9 s	0,5 à 1,0 s
75 %	16,1 s	21,1 s	22,2 s	0,5 à 0,9 s
50 %	51,8 s	62,1 s	73,2 s	0,9 à 4,0 s
25 %	69,6 s	87,4 s	98,4 s	0,4 à 2,1 s
0 %	51,0 s	62,1 s	74,4 s	0,5 à 4,5 s

Table 9. Durée médiane d'un lot de 30 requêtes.

Médiane par service	**1 serveur**	**Plages**	**Hachage**
F2 (personnalisée)	3,5 ms	7,2 ms	7,4 ms
F3 (personnalisée)	323 ms	377 ms	416 ms
F4 (exploration)	58 ms	119 ms	127 ms
F5 (exploration)	5 504 ms	6 596 ms	8 097 ms

Table 10. Services pris séparément.

Le serveur seul est le plus rapide dans les deux charges. Les configurations distribuées sont de 10 à 45 % plus lentes, le hachage derrière les plages : l'écart est net en exploration (62 s contre 74 s) et se perd dans le bruit en charge personnalisée. Une requête sur un lecteur ne touche qu'un shard dans les deux placements, mais elle traverse mongos, ce qui double le temps de F2.

Le travail est pourtant bien partagé : en exploration, chaque shard examine environ 25 millions de clés (24,4 et 26,4 millions par plages, 25,0 et 25,9 par hachage) contre 50,8 millions pour le

serveur seul. Le temps ne baisse pas pour autant. Nous pensons que les shards, le routeur et le programme Java se disputent les mêmes 8 cœurs et le même disque, et que mongos doit fusionner les résultats ; nous n'avons pas isolé ces causes. Le stockage passe de 494 Mo à 761 Mo (plages) et 849 Mo (hachage), chaque shard gardant ses propres index. Ces résultats ne disent rien d'un cluster de machines séparées.

Face à la prédiction, le bilan est mitigé. Pour la charge personnalisée, elle est juste : A gagne et F2 passe de 3,5 à 7,2 ms à cause du saut par mongos. Pour l'exploration, elle est fausse : nous attendions B et C proches de A ou un peu plus rapides, alors que B met 22 % de temps en plus et C 46 %. Le croisement prévu quand l'exploration progresse n'apparaît pas. L'absence de différence entre B et C se vérifie à peu près en charge personnalisée, pas en exploration. Quant au cache doublé que nous invoquions, nous ne l'avons pas mesuré.

7.3 E3. Qualité des recommandations

Le jeu d'évaluation regroupe les 3 dernières notes de lecteurs jamais chargés. 1 870 lecteurs sont éligibles, c'est-à-dire qu'au moins une de leurs notes cachées vaut 4 ou plus. Chaque liste compte 10 livres ; la référence propose les 10 livres les plus populaires que le lecteur n'a pas lus. Les valeurs sont des moyennes avec erreur type.

Méthode	**Précision@10**	**Rappel@10**	**Au moins un bon livre**	**Part du top 100**
F3, historique complet	0,0143 ± 0,0011	0,0615 ± 0,0045	11,0 %	9,3 %
F3, 5 premières notes	0,0052 ± 0,0006	0,0242 ± 0,0028	4,8 %	17,1 %
F3, 5 notes, sans tags	0,0051 ± 0,0006	0,0235 ± 0,0027	4,7 %	18,3 %
Popularité	0,0043 ± 0,0005	0,0183 ± 0,0024	3,8 %	100 %

Table 11. E3.

Avec l'historique complet, la précision de F3 vaut 3,3 fois celle de la popularité, et 11,0 % des lecteurs reçoivent au moins un bon livre, contre 3,8 %. Avec 5 notes, l'écart se resserre (0,0052 contre 0,0043) et les erreurs se recouvrent presque. Les valeurs absolues restent faibles : sur 10 000 livres, il y a peu de notes cachées à retrouver.

Les comptes de tags couvrent tout Goodreads et peuvent donc contenir la période cachée. Sans le profil de tags, le score des historiques tronqués passe de 0,0052 à 0,0051 : l'effet est faible sur cette mesure. Nous ne l'avons pas mesuré avec l'historique complet.

7.4 Vérification indépendante

Un script lit directement les fichiers CSV, sans base de données, et recalcule un résultat de F2 (32055 et 36002 : 84 livres communs, corrélation pondérée) et un de F4 (349 vers 7 : 427 et 2 419 lecteurs qui ont aimé, 137 en commun, 103,29 attendus, lift 1,326). Les valeurs coïncident avec celles de BookScope sur les deux déploiements.

7.5 E4. Un cas difficile pour F5

Pour dystopia et young-adult, F5 répond qu'il n'y a aucun lien au-delà du hasard : lift de 0,65, et de 0,39 sans les livres populaires. Pourtant 209 livres portent les deux tags, soit 64 % des 329 livres dystopia, dont The Hunger Games (11 065 et 25 968 comptes de tags). Seuls 25 lecteurs sur 9 999 sont attachés aux deux intérêts. Le résultat est trompeur : il ne parle que des lecteurs, alors que le lien visible dans le catalogue est fort.

Une cause se trouve dans le calcul. Les livres qui portent les deux tags sont retirés des deux côtés, mais leurs notes positives restent dans le total de chaque lecteur. Les lecteurs de dystopie aiment beaucoup ces livres : leur total contient des notes qui ne peuvent jamais compter comme young-adult, ce qui gonfle la valeur attendue et fait baisser le lift.

Nous avons changé cette seule décision : retirer aussi ces livres du total de chaque lecteur (option --adjust-shared). Les mêmes données et les mêmes 14 paires de tags sont mesurées avec l'ancienne et la nouvelle règle, et les résultats sont identiques sur le serveur seul et sur le cluster.

Paire	**Livres communs**	**Lecteurs pont**	**Lift**	**Verdict**
dystopia, young-adult	209	25 → 38	0,65 → 0,76	aucun lien, inchangé
mystery, thriller	1 098	121 → 302	0,95 → 1,41	aucun lien → persiste
fantasy, science-fiction	618	54 → 166	1,20 → 1,72	aucun lien → persiste
paranormal, romance	708	59 → 254	0,90 → 1,38	aucun lien → disparaît sans les cas populaires
vampires, romance	318	208 → 362	1,51 → 2,10	persiste, inchangé
horror, vampires	77	95 → 101	0,69 → 0,71	aucun lien, inchangé

Table 12. E4, avant et après le retrait des livres communs du total (extrait de 14 paires).

Pour dystopia et young-adult, le problème n'est pas résolu : le lift passe de 0,65 à 0,76 et le verdict reste « aucun lien ». Le calcul n'expliquait qu'une partie du résultat. Sur les 14 paires, le lift est multiplié par 1,23 en médiane (de 1,03 à 1,70), et d'autant plus que les deux tags partagent de livres : ×1,03 pour horror et vampires (77 livres communs), ×1,48 pour mystery et thriller (1 098), ×1,70 pour young-adult et fantasy (1 228).

La modification a un coût. Trois verdicts changent : fantasy et science-fiction (de 1,195 à 1,716) et mystery et thriller deviennent des liens qui persistent, et paranormal et romance devient un lien qui disparaît sans les cas populaires. Ces changements paraissent plausibles, mais nous n'avons pas de référence pour les confirmer, et le seuil de 1,2 avait été fixé à la main avec l'ancien calcul. Nous avons donc gardé l'ancien calcul par défaut ; la nouvelle règle reste disponible avec l'option.

Ce qui reste trompeur est la formulation de la réponse : « aucun lien au-delà du hasard » se lit comme « intérêts sans rapport », alors qu'elle ignore 209 livres communs. Signaler le recouvrement des tags dans le verdict serait la suite naturelle ; nous ne l'avons pas fait.

8. Limites

- Tout tourne sur une seule machine : les processus se partagent processeur et disque, donc la partie II ne dit pas ce que ferait un vrai cluster.
- Les shards n'ont pas de réplique : la perte d'un shard rend le service indisponible, même pour un lecteur de l'autre shard.
- E1 et E2 reposent sur 30 requêtes rejouées cinq fois : on obtient des écarts-types, pas des intervalles serrés.
- E3 : les valeurs sont faibles, et l'écart avec la popularité n'est pas net à 5 notes.
- Les seuils (50 voisins, 5 livres communs, 10 notes, tag générique à 50 %) ont été fixés à la main.
- La liste des tags de statut est incomplète : all-time-favourites, par exemple, est traité comme un tag de contenu.
- E4 : la correction testée n'efface pas le cas difficile, et les seuils de F5 n'ont pas été recalibrés.

Annexe : anomalies relevées dans la sélection

Chaque anomalie vient de data/prepared/report.json. L'identifiant donné en exemple permet de la retrouver dans le fichier source indiqué.

Anomalie	Nombre	Fichier	Exemple
Livres sans language_code	1 084	books.csv	book_id 45
Livres sans isbn	700	books.csv	book_id 106
Livres sans original_title	585	books.csv	book_id 75
Livres sans année de publication	21	books.csv	book_id 220
Titres partagés par plusieurs livres	33	books.csv	« 'Salem's Lot » : livres 349 et 1292
Paires (livre, tag) en double	8	book_tags.csv	ligne 211 : goodreads 3, tag owned (6 065 et 6 066)
Comptes de tags négatifs ou nuls	6	book_tags.csv	ligne 922 053 : livre 1935, compte -1
Tags de statut (to-read, owned, favorites...)	994	tags.csv, book_tags.csv	to-read sur 9 983 livres
Livres notés et dans to_read du même lecteur	460	ratings.csv, to_read.csv	lecteur 30126, livre 2

Table 13. Anomalies relevées.